

XGBoost

October 31, 2025

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from xgboost import XGBRegressor
from sklearn.model_selection import KFold, cross_val_score

sns.set(style="whitegrid", palette="muted", font_scale=1.2)
plt.rcParams["figure.figsize"] = (6, 4)
```

```
[2]: X_train = pd.read_csv("X_train.csv")
y_train = pd.read_csv("y_train.csv").squeeze()
X_test = pd.read_csv("X_test.csv")
y_test = pd.read_csv("y_test.csv").squeeze()

print("Train-test data loaded successfully!")
print(f"X_train: {X_train.shape}, X_test: {X_test.shape}")
```

Train-test data loaded successfully!
X_train: (780, 2), X_test: (195, 2)

```
[3]: # =====
# Define XGBoost Regressor (sample hyperparameters)
# =====
xgb_model = XGBRegressor(n_estimators=200,          # number of boosting rounds
    ↪ (commonly 100-1000)
    max_depth=3,          # tree depth controlling complexity (2-10 typical)
    learning_rate=0.1,    # step size shrinkage (0.01-0.3)
    subsample=0.8,        # fraction of training samples per tree (0.5-1.0)
    colsample_bytree=0.8,  # fraction of features per tree (0.5-1.0)
    gamma=0,              # minimum loss reduction for split (0-5)
    min_child_weight=1,    # min sum of weights in a child (1-10)
    random_state=42
)
```

```
[4]: # =====
# 10-Fold Cross-Validation on Train Set
# =====
kf = KFold(n_splits=10, shuffle=True, random_state=42)

r2_scores, rmse_scores, mae_scores = [], [], []

for train_idx, val_idx in kf.split(X_train):
    X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
    y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

    xgb_model.fit(X_tr, y_tr)
    y_pred_val = xgb_model.predict(X_val)

    r2_scores.append(r2_score(y_val, y_pred_val))
    rmse_scores.append(np.sqrt(mean_squared_error(y_val, y_pred_val)))
    mae_scores.append(mean_absolute_error(y_val, y_pred_val))

print("\n 10-Fold CV on Train Set:")
print(f"R2    → mean: {np.mean(r2_scores):.3f}, std: {np.std(r2_scores):.3f}")
print(f"RMSE → mean: {np.mean(rmse_scores):.3f}, std: {np.std(rmse_scores):.3f}")
print(f"MAE  → mean: {np.mean(mae_scores):.3f}, std: {np.std(mae_scores):.3f}")
```

```
10-Fold CV on Train Set:
R2    → mean: 0.407, std: 0.090
RMSE → mean: 1.122, std: 0.071
MAE  → mean: 0.886, std: 0.042
```

```
[5]: # =====
# Train final model on full training data
# =====
xgb_model.fit(X_train, y_train)
y_pred_train = xgb_model.predict(X_train)
y_pred_test = xgb_model.predict(X_test)

# =====
# Evaluate Train and Test Performance
# =====
def print_metrics(y_true, y_pred, dataset="Test"):
    r2 = r2_score(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    print(f"\n {dataset} Performance:")
    print(f"R2    = {r2:.3f}")
    print(f"RMSE = {rmse:.3f}")
```

```

print(f"MAE = {mae:.3f}")

print_metrics(y_train, y_pred_train, "Train")  # Corrected train metrics
print_metrics(y_test, y_pred_test, "Test")

```

Train Performance:

R^2 = 0.690
 RMSE = 0.822
 MAE = 0.650

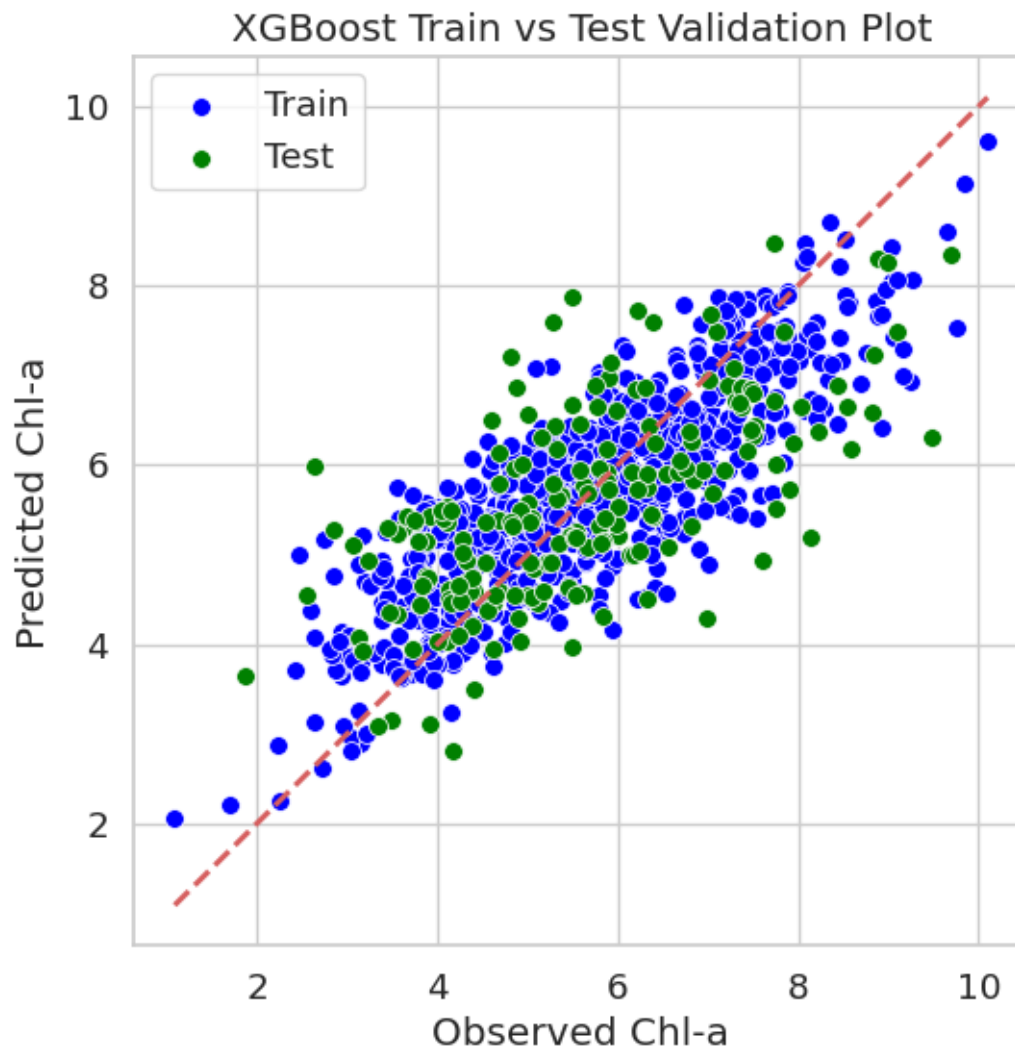
Test Performance:

R^2 = 0.432
 RMSE = 1.143
 MAE = 0.923

```

[6]: # =====
# Validation Plot
# =====
plt.figure(figsize=(6,6))
sns.scatterplot(x=y_train, y=y_pred_train, s=50, color='blue', label="Train")
sns.scatterplot(x=y_test, y=y_pred_test, s=50, color='green', label="Test")
plt.plot([min(y_train.min(), y_test.min()), max(y_train.max(), y_test.max())],
         [min(y_train.min(), y_test.min()), max(y_train.max(), y_test.max())],
         'r--', lw=2)
plt.xlabel("Observed Chl-a")
plt.ylabel("Predicted Chl-a")
plt.title("XGBoost Train vs Test Validation Plot")
plt.legend()
plt.show()

```



```
[7]: # =====
# Feature Importance
# =====
feat_importances = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': xgb_model.feature_importances_
}).sort_values(by='Importance', ascending=False)

print("\n Feature Importances:")
display(feat_importances)
```

Feature Importances:

Feature Importance

0	temp	0.791393
1	salt	0.208607

[]:

[]: